Graph Structure Is Necessary for Information Preservation Under Bounded Context

Patrick D. McCarthy

Abstract

Any system that (i) accumulates propositions about a world it cannot fully observe, (ii) operates under a bounded active context, a hard limit on the number of propositions simultaneously available for inference, and (iii) must preserve information as the proposition count grows beyond that bound, necessarily maintains a directed graph with typed edges over its propositions. We prove this claim from two independent arguments. The *information-preservation argument* shows that the only space-creating operation that does not lose information is factoring, extracting shared structure into named nodes with typed edges, and that factoring *is* graph construction. The *retention-dynamics argument* shows that any representation supporting sub-linear dependency queries, contextual retrieval, and selective removal with cascade necessarily encodes directed adjacency, which is a semantic graph regardless of substrate. We show that continuous-weight mechanisms (e.g., transformer self-attention) implement graph structure rather than constitute an alternative to it. The result applies to any system that stores propositions under bounded context, including artificial neural networks, institutional knowledge systems, and—to the extent that biological representations admit propositional decomposition—biological neural networks.

1 Introduction

As a knowledge base grows, the capacity to reason over it, at any single moment, does not grow proportionally. Biological working memory holds roughly 7 ± 2 items [18]. Transformer context windows, though large, degrade in inference quality as they fill with content irrelevant to the current problem [15]. Institutional decision makers cannot hold an entire organization's knowledge in a single meeting. In each case, there is a hard or effective bound on *active context*: the number of propositions simultaneously available for inference.

This paper asks a narrow question: **what are the requirements of a knowledge representation if information is not to be lost as the number of stored propositions exceeds the active context bound?**

The answer is a directed graph with typed edges. We prove this from two independent arguments, each sufficient on its own:

1. **Information preservation** (Proposition 1). When a bounded system must make space for new propositions, the only operation that creates space without losing information is *factoring*: extracting shared structure into a named node with typed edges. Factoring is graph construction. Discarding without adjacency information is blind; lossy compression without adjacency is blind discarding with extra steps.

2. **Retention dynamics** (Theorem 2). Any representation that supports sub-linear dependency queries, answering "what depends on this proposition?" in less than $O(n)$ time, necessarily stores directed adjacency. Selective removal with cascade (invalidating a proposition and propagating to its dependents) requires typed edges to scope correctly. These are the operations any system performs when maintaining knowledge under continuous evidence arrival.

The result is substrate independent. It does not prescribe an implementation (adjacency lists, edge tables, attention masks, synaptic connections). It proves that the *information content* of any adequate representation is a directed graph, regardless of how that graph is physically realized.

1.1 Related work

Graph-structured knowledge representations are widely used in practice: knowledge graphs [11], Bayesian networks [19], description logics [2], and the RDF/OWL stack [3] all represent knowledge as directed labeled graphs. Guarino's formal ontology [10] grounds this choice in the structure of the domain: entities, relations, and constraints map naturally to nodes, edges, and types. These demonstrate that graph structure is *effective* for knowledge representation. The question of whether it is *necessary*, as opposed to convenient, has received less formal attention.

The present paper provides the missing argument: under bounded active context, graph structure is not a design preference but a mathematical consequence of information preservation (Proposition 1) and sub-linear maintenance (Theorem 2). The typed directed edges of description logics and OWL are not an engineering convenience but the minimal structure any system must maintain. This result retroactively grounds decades of practice in a formal necessity argument.

Dupoux, LeCun, and Malik [8] propose a dual-system architecture (observation + action) with meta-control routing, motivated by cognitive science. Their framework is architectural and prescriptive: it argues that autonomous learning *should* integrate observation, action, and meta-control. The present work is complementary. Rather than proposing what intelligent systems should contain, we derive what they *must* contain under bounded context. The graph structure derived here also implies that information could flow bidirectionally between observation and action, not unidirectionally from perception to response.

The information bottleneck principle [23] establishes that optimal representations compress input while preserving information relevant to output. Our information preservation argument (Proposition 1) addresses a different question: not what to compress, but what structure the representation must have to compress without information loss.

Simon's bounded rationality [21] establishes that cognitive agents satisfice rather than optimize. Our bounded context constraint (C_n) formalizes a specific consequence: when the bound is hard, structure is not optional but necessary.

2 Formal Framework

We introduce the minimal definitions required for the two main results.

Definition 1 (World State Space)**.** Let W be a measurable space of possible world states. At time t, the true state $w_t \in W$ is not directly observable by any entity.

Definition 2 (Proposition and Knowledge Base)**.** A *proposition* p is a claim about W with an associated likelihood function $\ell_p : W \to [0, 1]$. A *knowledge base* K is a collection of propositions. The posterior over world states given K is:

$$P(w \mid K) \propto P_0(w) \cdot L_K(w)$$

where P_0 is a prior and L_K is the joint likelihood determined by K. Propositions in K may have arbitrary dependencies—shared structure, evidential interactions, causal relationships—so L_K need not factor as a product of individual likelihoods. (When propositions are conditionally independent given w, $L_K(w) = \prod_{p \in K} \ell_p(w)$; this is a special case, not assumed.) The existence of dependencies between propositions is precisely what motivates graph structure: if propositions were truly independent, dependency tracking would be unnecessary.

The *uncertainty* of K about w is the surprisal $U(w, K) = -\log P(w \mid K)$ [20, 5].

Definition 3 (Bounded Active Context)**.** For an entity n, let C_n denote its *active context capacity*: the maximum number of propositions simultaneously available for inference. Let $\mathrm{ctx}(t) \subseteq K$ be the set of propositions loaded into active context at time t, with $|\mathrm{ctx}(t)| \leq C_n$.

The *inference quality* for problem f is the relevance density:

$$\rho(f) = \frac{|K^{[f]}|}{|\mathrm{ctx}(t)|}$$

where $K^{[f]} \subseteq K$ is the set of propositions relevant to f. $\rho = 1$ when active context holds exactly the relevant propositions; $\rho \to 0$ when context is dominated by irrelevant content.

Definition 4 (Growth and Curation Regimes)**.** K has a maximum capacity K_{max} determined by substrate constraints. When $|K| < K_{max}$, K is in the *growth regime*: propositions accumulate freely. When $|K| = K_{max}$, K is in the *curation regime*: adding a new proposition requires making space.

Definition 5 (Contextual Retrieval)**.** Given problem f, *contextual retrieval* is the operation that loads $K^{[f]}$ into active context. The retrieval cost is the number of propositions examined to identify $K^{[f]}$.

Remark (Necessity via Selection)**.** The results in this paper show that graph structure is *cost-optimal* for information preservation under bounded context. A reviewer may ask: is it *necessary*, or merely sufficient? The answer depends on the competitive regime. In isolation, an entity could use any representation and survive—at higher cost. Under competition for finite resources, the cost differential becomes a survival differential: an entity paying $\Omega(n)$ per dependency query loses resources to a competitor paying $O(d)$ for the same query. Under sustained competition, cost-optimal structure is selected for, and alternatives are eliminated. Throughout this series, "necessary" means necessary for persistence under competition—selective necessity, not logical impossibility of alternatives.

3 Graph Necessity from Information Preservation

Definition 6 (Factoring)**.** Let $p_1, \ldots, p_m \in K$ share mutual information: there exists a random variable F^* (a function of W) such that $I(F^*; \ell_{p_i}) > 0$ for all i, and the joint likelihood decomposes as

$$\ell_{p_i}(w) = g_i\big(\ell_{p_i}^{\mathrm{res}}(w),\ F^*(w)\big)$$

for some residual $\ell_{p_i}^{\text{res}}$ and reconstruction function g_i. The decomposition need not be unique; any valid decomposition suffices. In the general case, g_i may be an arbitrary measurable function: the definition does not require multiplicative, additive, or any other restricted functional form. The permissiveness is deliberate: the theorem's force comes not from restricting g_i but from the requirement that g_i be *stored* (as a typed edge) to enable reconstruction. The weaker the constraint on g_i, the stronger the conclusion that typed edges are necessary. *Factoring* is the operation that extracts F^* into a single named, reusable unit and replaces the shared component in each p_i with a link to F^*.

This definition is deliberately minimal. It specifies only that a shared component is extracted and linked. It does not specify that the links are directed, that they carry type information, or that the result has graph structure. Proposition 1 derives these properties from the requirement of posterior preservation: direction and typing are consequences of reconstruction, not stipulations of the definition.

Remark. Factoring is not a novel operation introduced by this paper. It is the central operation of description logics [2]: the TBox/ABox separation extracts shared conceptual structure (the TBox) from individual assertions (the ABox), and concept subsumption hierarchies are literally shared components referenced by multiple individuals. Upper ontologies such as BFO [1] and DOLCE [14] carry this further: approximately 35 categories in BFO organize all of biomedical knowledge, demonstrating that real-world knowledge bases have abundant shared structure amenable to factoring. The definition above formalizes what these traditions have practiced for decades.

Definition 7 (Information Preservation)**.** A *space-creating operation* $f : K \to K'$ reduces active context load: $\text{load}(K') < \text{load}(K)$, where load counts the propositions simultaneously required in active context. (Node count may increase—factoring adds a shared node—but context load decreases because redundant copies are replaced by shared references.) Such an operation is *information-preserving* if the posterior over world states is unchanged:

$$P(w \mid K') = P(w \mid K) \quad \text{for all } w \in W.$$

This is posterior preservation: the operation may reorganize, compress, or restructure propositions, but the joint constraint that K places on W is not weakened.

Proposition 1 (Graph Necessity from Information Preservation)**.** *Let K be a knowledge base with bounded active context C_n (Definition 3) in the curation regime (Definition 4). Any information-preserving space-creating operation on K (Definition 7) as $|K|$ grows beyond C_n necessarily produces named units with directed, typed references between them—i.e., a directed graph with typed edges.*

Argument. In the curation regime, adding any new proposition p_{new} requires making space. Any space-creating operation $f : K \to K'$ with $\text{load}(K') < \text{load}(K)$ falls into exactly one of two categories: either it preserves the posterior (Definition 7), or it does not. We show that every information-preserving option produces graph structure, and that non-preserving options require graph structure to avoid uncontrolled information loss.

Option A: Discard an existing proposition. Without explicit relational structure, the entity has no mechanism to determine which existing propositions are redundant. Two propositions p_1, p_2 that share a latent factor appear as independent entries; there is no stored evidence that they share structure. The choice of which to discard is therefore made without information about relational dependencies. Discarding p_{old} loses the information it encodes, and potentially loses the information that p_{old} has any relation to other propositions, a second-order loss. This is information loss.

A natural objection: per-proposition metadata (access frequency, recency, embedding similarity to recent queries) can inform the discard decision without stored adjacency. This is true for a single proposition in isolation. But propositions share structure, and metadata must reflect that sharing to be accurate: the recency of p_1 is relevant to the discard decision for p_2 when both ground the same action. Sharing metadata across propositions requires storing which propositions share which metadata, which is storing directed relationships between entries. The metadata-based alternative converges to adjacency the moment it must account for shared structure, which is the non-degenerate case (Remark on shared structure, below).

Option B: Factor: extract shared structure (Definition 6). If propositions share a common informational component F^*, factoring extracts F^* as a single named node with typed edges, replacing redundant copies with shared references. Space is created without loss: the original information is reconstructable from F^* plus the residuals. Factoring also resolves the update problem: when new evidence revises F^*, the single node is updated once, and the revision propagates to all propositions that reference it via edges: $O(1)$ instead of $O(|K|)$.

By Definition 6, factoring produces nodes and directed typed edges. This is graph construction.

Option C: Reorganization: rewrite propositions into a more compact equivalent encoding. Propositions $p_1, \ldots, p_m$ could be replaced by $q_1, \ldots, q_k$ ($k < m$) that are jointly equivalent: the same posterior over W is induced. This is a change of basis in the information-theoretic sense.

If the reorganization preserves all dependency relationships, which propositions ground which, then the result is an isomorphic knowledge base with fewer nodes. But "preserving dependency relationships" requires storing those relationships, which is storing adjacency. If the reorganization does *not* preserve dependency relationships, then downstream propositions that depended on the original p_i are silently broken: they reference nodes that no longer exist, with no forwarding information. The entity cannot determine what broke without adjacency.

There is a special case: reorganization that extracts shared structure across the p_i into shared nodes referenced by the q_j. This is factoring (Option B). Reorganization that does not extract shared structure but merely rewrites individual propositions more compactly does not create space from shared structure; it creates space from per-proposition compression, which is lossless compression (Option D).

A stronger form of reorganization is *ontology change*: replacing the proposition vocabulary entirely. An entity with 100 propositions about individual animals might reconceptualize to 10 propositions about species. The new propositions are not factored components of the old ones; they are a different representational vocabulary. However, ontology change does not escape the theorem. It is a one-time translation that produces a new knowledge base K'. The new K' still describes a structured world, still operates under bounded C_n, and still accumulates propositions as evidence arrives. As $|K'|$ grows, the propositions in K' develop shared structure—the species propositions share habitat, physiology, evolutionary lineage—and the same curation pressure applies. Ontology change is analogous to random reseeding in optimization: it selects a different starting point but does not change the landscape. The landscape is bounded context over shared-structure propositions, and every ontology that describes a structured world converges to the same factoring pressure.

Option D: Lossless compression: encode propositions more compactly without information loss. Lossless compression reduces the physical footprint of stored propositions, and any system should compress to the extent possible. But compression has a floor: information has an irreducible size (Shannon entropy, Kolmogorov complexity). Once that floor is reached, the active context bound C_n remains. New propositions still arrive. The bound is a constraint on how many propositions can be *simultaneously active for inference*, not on how compactly they are encoded in

storage. A compressed bitstring must be decompressed before any proposition can be accessed or any dependency traced; the decompressed form faces the same curation pressure as the original. Compression is necessary but does not solve the structural problem the proposition addresses.

Option E: Lossy compression: replace propositions with a summary that occupies less space. Compressing p_1 and p_2 into a summary p_s that discards detail is a form of discarding: the lost detail may ground other propositions or actions. If p_1 has dependents, propositions that reference it, then replacing p_1 with p_s silently invalidates those dependents unless the entity knows what depends on the lost detail. But knowing what depends on p_1 *is* knowing p_1's adjacency: its edges. Without stored adjacency, lossy compression is blind discarding: the entity cannot evaluate what breaks. With stored adjacency, the entity can scope the damage, notify dependents, and compress safely; but storing adjacency is maintaining a graph. Lossy compression that preserves correctness requires the same relational structure as factoring. Lossy compression that ignores relational structure is Option A with extra steps.

Exhaustiveness. The partition above is complete because any $f : K \to K'$ with $|K'| < |K|$ either preserves the posterior (Definition 7) or does not. If it does not, it is lossy (Options A, E). If it does, it must produce K' with fewer propositions but the same joint likelihood. The ways to achieve this are: extract shared structure into reusable units (B), rewrite to an equivalent encoding with fewer propositions (C), or compress the physical representation without changing propositional count (D). These three exhaust the posterior-preserving options because any such operation either reduces the number of distinct likelihood factors (B, C) or reduces per-factor storage cost (D); there is no fourth category, since the only degrees of freedom are how many factors there are and how each is stored. Of these, D is orthogonal (it addresses storage density, not the active-context bound). C either reduces to B (when shared structure is extracted) or requires stored adjacency to preserve dependencies. Only B (factoring) creates space, preserves the posterior, and solves the structural problem.

On the limits of exhaustiveness. The partition above is argued, not proved in the formal sense, because the space of possible operations on a knowledge base is not itself formally bounded. We cannot enumerate all conceivable data structures or operations. However, this limitation does not weaken the overall result, because Theorem 2 provides an independent argument that does not rely on exhaustiveness.

The falsifiable content of the theorem is not "the representation is a graph"—which risks being unfalsifiable if *graph* is defined broadly enough to include any structure that works. The falsifiable content is the *cost prediction*: Theorem 2 establishes that answering "what depends on p?" costs $\Omega(n)$ without stored directed adjacency and $O(d)$ with it. This is a measurable, empirical claim. A system that achieves sub-linear dependency query without storing any form of directed relationship between propositions—not adjacency lists, not sparse matrices, not learned attention patterns, not any structure from which pairwise dependency can be read in $o(n)$ time—would falsify Theorem 2. The theorem does not say "everything is a graph." It says there is a specific cost for not storing directed adjacency, and that cost is $\Omega(n)$ per query.

Posterior preservation forces directed, typed structure. Definition 6 specifies only that F^* is extracted and linked. We now derive the structural properties that posterior preservation (Definition 7) requires of these links.

Nodes. The named units—shared components F^* and residual propositions p_i^{res}—are the nodes. Each is an identifiable, individually addressable unit of information.

Direction is required by reconstruction. To recover the original likelihood $\ell_{p_i}(w) = g_i(\ell_{p_i}^{\text{res}}(w), F^*(w))$, the system must know that p_i depends on F^*—not vice versa. This asymmetry is not stipulated

but entailed: revising F^* invalidates every p_i's reconstruction and requires re-evaluation; revising p_i does not invalidate F^*. A system that stores only symmetric (undirected) links cannot determine which units require re-evaluation when one unit changes—it must re-evaluate in both directions, degrading to worst-case cascade. The reconstruction path *is* the direction: from shared component to dependent proposition.

Typing is required by correct reconstruction. Knowing that p_i depends on F^* is insufficient for reconstruction. The system must also store *how* F^* enters the reconstruction: the function g_i. Different propositions depend on the same F^* in different ways—F^* may *ground* p_1 (truth value depends on F^*), *evidence* p_2 (likelihood is informed by F^*), or *cause* p_3 (state is produced by F^*). Without the stored g_i, the system cannot recover ℓ_{p_i} from F^* and the residual, and posterior preservation fails. The reconstruction function g_i is the edge type: it distinguishes the *kind* of dependency. This is not a design choice but a requirement—without it, the factoring is information-losing.

Typing also scopes cascade. When F^* is invalidated, typed edges enable cascade to follow only dependency-bearing links, keeping cost proportional to $|D_{F^*}|$ (the true dependency set) rather than $|\mathrm{adj}(F^*)|$ (all neighbors). This is a consequence of having the type, not an additional requirement.

A collection of named units with directed, typed links between them is a directed graph with typed edges, regardless of whether it is implemented as adjacency lists, pointers, sparse matrices, or any other medium. The graph structure is derived from the reconstruction requirement, not stipulated in the definition of factoring.

Remark on shared structure. The proposition requires that propositions share informational components. This is not an additional assumption but a consequence of the knowledge base describing a structured world. The proposition "fire is hot" is relevant to cooking, camping, metallurgy, and fire safety; any knowledge base spanning these domains contains this shared component. A knowledge base with *zero* shared structure would be a collection of completely independent facts about completely unrelated domains: each proposition grounded in its own private ontology with no overlap. Such a knowledge base faces no curation pressure (nothing is redundant, so nothing can be factored) and requires no structural optimization. The proposition addresses the non-degenerate case: knowledge bases about structured worlds, which is every knowledge base that admits inference across domains. The success of upper ontologies confirms that this is the typical case: BFO [1] organizes all of biomedical science under approximately 35 shared categories; DOLCE [14] provides cross-domain foundational categories reused across engineering, geography, and law. These are not design choices but reflections of shared structure that already exists in the domains they describe.

Remark on compression. Lossless compression changes how compactly propositions are stored, not how many can be simultaneously active. The bounded-context premise ($|K| > C_n$) is a constraint on active inference capacity, not storage density. No encoding scheme removes this bound. Compression is necessary but insufficient: it delays the curation problem without eliminating it.

4 Graph Necessity from Retention Dynamics

We first derive the operations that any knowledge maintenance system must support, then prove that any representation supporting them necessarily encodes directed adjacency.

Lemma 1 (Necessary Maintenance Operations)**.** *Let K be a knowledge base under bounded active context C_n (Definition 3), with continuous evidence arrival, where evidence can be corrupted or invalidated. Then K must support the following four operations:*

(i) ***Addition with provenance.***

(ii) ***Contextual retrieval.***

(iii) ***Dependency query.***

(iv) ***Selective removal with cascade.***

Proof. None of the three premises (bounded context, evidence arrival, corruptibility) mentions adjacency or graph structure. We derive each operation from these premises alone.

(ii) from bounded context. $|K| > C_n$ means the entity cannot load all propositions simultaneously. For any problem f, the entity must select which propositions to load. If selection is random or exhaustive, inference quality $\rho(f) \to 0$ as $|K|$ grows (Definition 3). Therefore the entity must identify and load $K^{[f]}$, the relevant subset. This is contextual retrieval.

(i) from evidence arrival. Propositions do not arrive in isolation. A new proposition p is grounded in existing knowledge: the evidence for p references existing propositions. If p is added without recording what it was grounded in, then when those grounds are later revised, the entity has no mechanism to determine that p is affected. The entity must store, at insertion time, which existing propositions p depends on. This is addition with provenance.

(iii) and (iv) from corruptibility. Evidence can be wrong. When proposition p is invalidated by new evidence, every proposition whose truth value depends on p is potentially compromised. The entity must determine which propositions are grounded through p (dependency query) and re-evaluate or remove them transitively (selective removal with cascade). Without these operations, a single corrupted proposition silently invalidates an unbounded number of dependents, and the entity has no mechanism to detect or scope the damage. □

Theorem 2 (Graph Necessity from Retention Dynamics)**.** *Let K be a knowledge base supporting the maintenance operations derived in Lemma 1. Restated for reference:*

(i) ***Addition with provenance****: insert a proposition p together with its evidential grounds: which existing propositions p depends on, and by what relation.*

(ii) ***Contextual retrieval****: given problem f, retrieve exactly $K^{[f]}$.*

(iii) ***Dependency query****: given proposition p, determine which propositions are grounded through p.*

(iv) ***Selective removal with cascade****: when p is invalidated, remove p and re-evaluate every proposition whose grounding depends on p, transitively, until K stabilizes.*

Any representation achieving sub-linear cost on all four operations simultaneously necessarily encodes directed adjacency between propositions. Under bounded active context C_n, directed adjacency converges to typed edges—carrying the kind of dependency on each edge—because untyped cascade wastes context on irrelevant re-evaluation. A directed graph with typed edges over propositions is a semantic graph regardless of implementation.

Proof. **Part 1: Dependency queries are necessary and continuous.** Knowledge maintenance operates continuously: every new piece of evidence potentially shifts the status of existing propositions. When evidence revises or invalidates a proposition p, the system must evaluate every proposition whose grounding passes through p. This is not a batch operation that can be

deferred; acting on a proposition whose ground has been invalidated risks incorrect inference. The dependency query "what is grounded through p?" is therefore invoked at the frequency of evidence arrival.

Part 2: Lower bound without adjacency. Consider a representation R that stores n propositions without explicit directed relationships between them (a flat store, hash table, embedding space, or any structure that does not record which propositions depend on which). To answer the dependency query for p, R must examine each of the other $n-1$ propositions to determine whether its grounding involves p. Without stored adjacency, there is no mechanism to restrict this examination: any proposition might depend on p, and the only way to determine this is to check.

In any model where the representation does not store explicit inter-proposition relationships (cell-probe, comparison-based, or any model where dependency is not pre-computed), the cost of a single dependency query is therefore $\Omega(n)$. Since dependency queries are invoked at evidence-arrival frequency (Part 1), the amortized cost of maintenance is $\Omega(n)$ per evidence event, growing linearly with $|K|$.

Part 3: Cascade amplifies the cost. Selective removal is not a single deletion. When p is removed, each proposition q grounded through p must be re-evaluated: does q have alternative grounds, or should it also be removed? If q also fails, its dependents must be re-evaluated in turn. This is a transitive closure on the dependency relation.

With explicit adjacency, cascade follows edges: cost is $O(|D_p|)$ where $|D_p|$ is the transitive dependency set. Without adjacency, each level of cascade requires an $\Omega(n)$ scan. Total cost: $\Omega(n \cdot L)$ where L is the cascade depth, versus $O(|D_p|)$ in the graph where $|D_p| \ll n$ for typical propositions with bounded connectivity.

Remark. Truth maintenance systems [7, 6] are a direct implementation of operations (iii) and (iv). A TMS stores directed justifications (which propositions ground which) and propagates belief revision through them when any justification is invalidated, precisely the dependency query and cascade operations this theorem proves are necessary. The fact that AI research independently invented TMS to solve this exact problem is empirical evidence for the theorem's claim: without stored directed adjacency, maintenance under evidence arrival is intractable.

Part 4: Any sub-linear solution encodes adjacency. Suppose R achieves $o(n)$ cost for dependency queries. Then R must store, for each proposition, information about which other propositions depend on it; otherwise the $\Omega(n)$ lower bound of Part 2 applies. This stored information is a set of directed relationships: for each proposition p, a record of the propositions that reference p as part of their grounding.

This is precisely a directed adjacency structure. The nodes are propositions. The edges are directed dependency relations. Any representation that stores this information, regardless of whether it is implemented as adjacency lists, edge tables, pointers, or any other mechanism, *is* a directed graph over propositions.

Remark on approximate methods. Approximate retrieval mechanisms, learned embeddings, locality-sensitive hashing, approximate nearest-neighbor indices, achieve sub-linear performance on operation (ii) (contextual retrieval) without storing explicit directed adjacency. This does not circumvent the theorem, because the theorem requires sub-linear cost on all four operations *simultaneously*. Approximate similarity over an embedding space can identify propositions *related to* a query, but cannot answer "what is *grounded through* p?" (operation iii) or propagate invalidation transitively through dependency chains (operation iv). These operations require knowing which propositions depend on which, not which are *nearby* in a metric space. Proximity and dependency are distinct relations: p and q may be embedding-close yet inferentially independent, or

embedding-distant yet linked by a grounding chain. Any representation that achieves sub-linear cost on dependency queries and cascade necessarily stores directed dependency relationships, which is directed adjacency.

Remark on the scope of "graph." The claim is not that the representation must have some structure (which is trivially true of any non-random encoding). The claim is that the representation must support scoped cascade under corruption (Lemma 1), and that the minimal structure satisfying this requirement is directed typed adjacency. Representations that store structure but not directed dependency, flat stores with metadata, embedding spaces with similarity metrics, hash tables with access counts, fail operation (iii): they cannot answer "what is grounded through p?" without $\Omega(n)$ scan. The graph is not defined broadly and then shown to be necessary. The operations are derived from premises that do not mention graphs (Lemma 1), and the graph is what falls out.

Part 5: Typed edges are the convergence point under bounded context. Parts 1–4 establish that directed adjacency is necessary: without it, dependency queries cost $\Omega(n)$. This is a hard lower bound. Typed edges are a separate claim at a different level of necessity.

A bare adjacency structure (untyped edges) achieves sub-linear dependency queries. It is not incorrect. But when p is invalidated, untyped cascade cannot distinguish propositions *inferentially grounded* through p (which require re-evaluation) from propositions merely *associated* with p (which do not). Every neighbor must be treated as potentially dependent, expanding cascade to the full neighborhood $|\text{adj}(p)|$ rather than the true dependency set $|D_p|$.

Under bounded C_n, this expansion is not merely wasteful—it consumes the same scarce resource (active context slots) that the entity needs for inference. Each unnecessary re-evaluation during cascade loads propositions into context that displace problem-relevant content, degrading ρ for concurrent inference. As $|K|$ grows and $|\text{adj}(p)|$ diverges from $|D_p|$, untyped cascade consumes an increasing fraction of C_n on irrelevant re-evaluation. Typed edges—carrying the relation type ("grounds," "evidences," "causes," "supports")—restrict cascade to dependency-bearing edges, keeping cost proportional to $|D_p|$.

The distinction matters: directed adjacency is *necessary* (no alternative achieves sub-linear maintenance); typed edges are what directed adjacency *converges to* under bounded-context pressure. A system with untyped edges functions but pays an increasing tax on C_{free} as knowledge grows. This is the same convergence pattern—not logical necessity, but what bounded systems converge to under C_n pressure—that recurs whenever a finite-capacity system must scale.

Remark on empirical convergence. The history of knowledge representation confirms this trajectory. Early semantic networks used untyped "is-related-to" links; Woods [24] demonstrated that these were semantically ambiguous and operationally inadequate: the system could not distinguish inheritance from association, causation from correlation. The field converged to typed relations: RDF introduced typed predicates [3], OWL distinguished object properties, data properties, and annotation properties with transitive, symmetric, and inverse role characteristics [2]. This trajectory—starting untyped, encountering scaling failure, converging to typed—is the progression Parts 1–5 predict.

A directed graph with typed edges over propositions is a semantic graph. □

Remark (Cascade as error recovery). The cascade mechanism proven necessary in Parts 3–5 is not only an efficiency requirement. It is the error recovery mechanism. When evidence corrupts a proposition p, the system must identify every proposition whose grounding passes through p, re-evaluate or remove each, and scope the damage to the actual dependency set D_p. Without

stored typed adjacency, this scoping is impossible: the system cannot distinguish propositions affected by the corruption from those that are not, and must treat the entire knowledge base as potentially contaminated. Stored typed adjacency reduces error recovery from $\Omega(n)$ (full re-evaluation) to $O(|D_p|)$ (dependency-scoped re-evaluation). The maintenance structure and the robustness structure are the same structure.

Corollary 3 (Edge Growth Dominates Node Growth)**.** *As K matures under continuous evidence arrival, the ratio of edges to nodes increases. Each new proposition p_{new} added to K arrives with at least one edge (its evidential ground, by operation (i) of Theorem 2). As existing propositions are cross-grounded through new evidence, new edges are added between existing nodes without adding new nodes. Edges are the cheapest structure in the graph: each edge is a typed pointer between existing nodes, not new propositional content. The information density of the graph, the ratio of relational information to propositional content, therefore increases as K matures. A mature knowledge base is edge-dense: most of its informational value is in the relationships between propositions, not in the propositions themselves.*

5 Continuous-Weight Mechanisms Are Graph Implementations

A natural objection: modern attention mechanisms achieve contextual retrieval without explicit graph construction. Does this constitute a non-graph alternative?

Theorem 4 (Contextual Retrieval Requires Adjacency Computation)**.** *Any mechanism that performs contextual retrieval over n propositions, selecting which propositions are relevant to a given query, must compute pairwise relevance between the query and stored propositions. This computation either (a) evaluates all n propositions per query at $O(n)$ or $O(n^2)$ cost (dynamic adjacency), or (b) pre-computes and stores relevance relationships at $O(d)$ per query where d is the local degree (stored adjacency). In case (b), the stored relationships constitute a directed graph. Case (a) is the cost of recomputing that graph from scratch at every query.*

Proof. Contextual retrieval for problem f requires identifying $K^{[f]}$, the propositions relevant to f. This requires determining, for each proposition $p \in K$, whether p is relevant to f. Without stored information about relevance relationships, every proposition must be evaluated: cost $\Omega(n)$ per query.

Full self-attention is the canonical example: $A_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d_k})$ evaluates all n^2 pairwise relevances per query. This is not "computing a graph" in the trivial sense that any matrix can be called a graph; it is performing the specific computation (pairwise relevance determination) that a pre-built graph would make unnecessary. The attention matrix is dense: all n^2 entries are nonzero, encoding no structural information. The computational cost is the cost of having no stored structure.

Pre-computed adjacency, stored information about which propositions are relevant to which queries, reduces retrieval to $O(d)$ where d is the number of stored relevant relationships. This is a directed graph: nodes are propositions, edges are relevance relationships, and the direction encodes which proposition informs which query context.

Sparse attention mechanisms, learned sparsity patterns, local windows, routing functions, reduce the $O(n^2)$ cost by learning which pairwise relevances are worth computing: they are learning which edges to store. The KV cache materializes computed relevances for reuse across queries.

Each optimization converts dynamic adjacency computation into stored adjacency, progressively building the graph that Proposition 1 and Theorem 2 prove is necessary.

The result is not that "attention is a graph" (which would be trivially true of any matrix) but that contextual retrieval requires adjacency information, and the only question is whether that information is computed dynamically ($O(n^2)$) or stored explicitly ($O(d)$). Stored adjacency over propositions with typed relevance relationships is a directed graph. □

Remark (Scaling context windows). Growing C_n directly does not solve the retrieval problem. For any specific problem f, the relevance density $\rho(f) = |K^{[f]}|/|\mathrm{ctx}(t)|$ (Definition 3) decreases as context fills with content not relevant to f, an effect documented empirically as "lost in the middle" [15]. The efficient path is not to grow the context window but to grow the encoded knowledge accessible via stored adjacency: filling the graph, not the context window.

6 Discussion

The two results establish graph necessity from independent premises. Proposition 1 requires only the curation regime (bounded capacity, new propositions arriving). Theorem 2 requires only continuous knowledge maintenance (evidence arrives, dependencies must be traced) and does not assume lossless preservation ; it applies equally to systems that tolerate information loss, because the cost argument concerns maintenance operations, not retention policy. Any system subject to both constraints, which includes every physical system that accumulates knowledge, is doubly committed to graph structure.

6.1 The continuous representation objection

The strongest objection to this paper's results is that they assume discrete propositions, while the most capable knowledge systems , large language models, biological neural networks, store knowledge in continuous distributed representations that do not decompose into identifiable propositions. If there are no discrete propositions, there is nothing to factor (Proposition 1), nothing to query dependencies over (Theorem 2), and calling an attention matrix a "graph" (Theorem 4) is a mathematical relabeling, not a structural discovery. The objection: the theorems prove graph necessity for an idealized system that no successful architecture actually implements.

This objection is serious and requires a substantive response. We offer four.

(1) Continuous representations do encode discrete propositions. The continuity of the substrate does not entail that the information content is non-propositional. Meng et al. [17] demonstrate that specific factual associations in GPT-class models can be localized, edited, and ablated at identified parameter subsets: "The Eiffel Tower is in Paris" has a locatable representation that can be revised to "The Eiffel Tower is in Rome" without retraining. Geva et al. [9] show that feed-forward layers in transformers function as key-value memories, with individual neurons activating for identifiable semantic patterns. Li et al. [13] identify discrete computational circuits within continuous networks for mathematical reasoning.

These results establish that continuous weight matrices encode propositional content, not approximately or metaphorically, but in a sense precise enough to support targeted editing. The propositions are *superposed* (multiple propositions share parameters) and *distributed* (each proposition

spans multiple parameters), but they are identifiable, editable, and individually removable. Superposed encoding is precisely the condition that makes factoring valuable: shared structure across propositions is the input to factoring, and continuous networks contain it abundantly.

(2) The claim is about information content, not implementation. Proposition 1 and Theorem 2 prove that the *information content* of any adequate representation has graph structure. They do not prescribe adjacency lists, edge tables, or symbolic labels. A neural network whose weights encode directed dependencies between propositions *is* maintaining a graph in the sense of Definition 2: the graph exists in the information, not in the data structure.

The objection that "if any structured representation counts as a graph, the claim is trivially true" misidentifies the contribution. The claim is not that representations have structure (which is trivial) but that the structure must be *directed*, *typed*, and *adjacency-preserving*, and that representations lacking these specific properties lose information or pay $\Omega(n)$ maintenance costs. Not all structure is graph structure. A flat embedding space has structure (metric, topology) but not directed adjacency; a hash table has structure (key-value mapping) but not typed dependencies. The theorems identify *which* structural properties are necessary, not merely that some structure is required.

(3) Catastrophic forgetting is the theorem's prediction for systems without explicit adjacency. This is the strongest empirical evidence for the result. Theorem 2 proves that without stored directed adjacency, selective removal with cascade, updating one proposition and propagating correctly to its dependents, costs $\Omega(n)$ per evidence event. The prediction: systems that store propositions in continuous weights without explicit adjacency should exhibit *uncontrolled interference when knowledge is updated*.

This is precisely catastrophic forgetting [16, 12]: updating a neural network on new information degrades previously learned information unpredictably. The network cannot scope the update because it lacks the adjacency to determine what depends on the modified parameters. Continual learning remains an open problem in deep learning not because of insufficient engineering but because the underlying representation lacks the structural property the theorem identifies: directed adjacency that enables scoped updates.

The response to the continuous representation objection is therefore not defensive but *predictive*: the theorem identifies the specific structural deficit (missing explicit adjacency), predicts the specific failure mode (unscoped updates causing cascading interference), and the most successful continuous representation systems exhibit exactly that failure mode. The objection that the theorems do not apply to neural networks is refuted by the fact that neural networks suffer from precisely the pathology the theorems predict for representations without graph structure.

(4) Knowledge graph embeddings are continuous implementations of graph structure. A further data point: knowledge graph embedding methods (TransE [4], RotatE [22]) project graph structure into continuous vector spaces. The embeddings are continuous, but the information content they encode, which entities are related to which, by what relation, is directed typed adjacency. Inference tasks such as link prediction require reconstructing adjacency from the embedding space. These methods do not eliminate graph structure; they implement it in a continuous substrate, which is exactly the relationship Proposition 1 and Theorem 2 predict between information content (graph) and implementation (free parameter).

6.2 Scope and non-claims

The theorems prove that the *information content* of the representation must be a directed graph. They do not prescribe a specific graph formalism (property graph, RDF, Bayesian network) or

implementation (adjacency lists, sparse matrices, attention masks). The graph is the abstract structure. The substrate is a free parameter.

Implications for AI architecture. Theorem 4 reframes the relationship between graph-based and neural approaches: they are not competing paradigms but different implementations of the same structural necessity. Systems that compute adjacency dynamically (full attention) pay $O(n^2)$; systems that store it explicitly (graphs, sparse attention, KV caches) pay $O(d)$. The trend in architecture design, from dense attention toward sparse, structured, and cached retrieval, is convergence toward explicit graph structure, driven by the same cost pressures the theorems identify.

The result suggests that hybrid architectures combining explicit knowledge graphs with neural retrieval are not an engineering convenience but a response to a structural constraint: as $|K|$ grows, systems must encode adjacency to maintain inference quality under bounded context.

Free context and the purpose of graph structure. The graph structure derived in this paper exists to serve a single resource: *free context*, the portion of C_n not consumed by maintenance of existing knowledge and available for novel problems. Formally, $C_{\text{free}}(t) = C_n - |\text{ctx}(t)|$, where $|\text{ctx}(t)|$ is the number of context slots occupied by currently active propositions. Factoring (Proposition 1) increases C_{free} by compressing shared structure into fewer propositions; stored adjacency (Theorem 2) increases C_{free} by enabling targeted retrieval—loading only the d relevant propositions rather than scanning all n—so that fewer context slots are consumed by irrelevant material during inference. Every result in this paper is, at root, a mechanism for maximizing C_{free}—the context capacity available for novel problems after existing-knowledge maintenance.

Implications for scaling. The scaling remark following Theorem 4 challenges the prevailing approach of growing context windows as the primary path to more capable inference. Under bounded context, larger windows with undifferentiated content *reduce* inference quality for any specific problem. The alternative, encoding knowledge into a graph and retrieving the relevant subgraph per problem, maintains $\rho = 1$ regardless of total knowledge size. This is the difference between scaling the window and scaling the knowledge.

6.3 Falsifiable predictions

The theorems yield seven predictions that are empirically testable. Each states a consequence of the formal results and the observation that would falsify it.

1. Catastrophic forgetting is structural, not parametric. Theorem 2 predicts that systems storing propositions in continuous weights without explicit adjacency exhibit uncontrolled interference when knowledge is updated. This is catastrophic forgetting. The prediction: continual learning will not be solved by regularization, replay, or parameter isolation alone. Any solution that works will encode directed adjacency over propositions, whether explicitly or implicitly. A system that solves continual learning without adjacency would falsify Theorem 2.

2. Architectural convergence toward stored adjacency. Theorem 4 proves that contextual retrieval without stored adjacency costs $O(n^2)$; with stored adjacency, $O(d)$. The prediction: the dominant trajectory in neural architecture design will move from dense attention toward sparse, structured, cached retrieval, progressively building explicit adjacency. Sparse attention patterns, learned routing, KV caches, and retrieval-augmented generation are early instances. More specifically, learned sparsity patterns will increasingly resemble stored dependency structure: current routing in mixture-of-experts is content-based; the prediction is that effective routing will become topology-aware, because that is what the cost pressure selects for. A system that achieves sub-linear contextual retrieval without encoding any form of directed adjacency would falsify Theorem 4.

3. Context-optimizing architectures will outcompete context-expanding architectures on cost. Current large language models maximize functional context by expanding the context window. Theorem 4 shows this cannot scale: relevance density ρ degrades as undifferentiated content fills the window, and the retrieval cost remains $O(n^2)$. An architecture that accrues structured knowledge and retrieves the relevant subgraph per problem maintains $\rho = 1$ at $O(d)$ cost regardless of total knowledge size. The prediction: as task complexity and knowledge volume grow, systems that optimize context (structured accrual + selective retrieval) will undercut systems that expand context (larger windows + denser packing) on cost per correct answer, and competitive pressure will drive convergence toward the former. A context-expanding architecture that maintains cost parity with structured retrieval at arbitrary knowledge scale would falsify Theorem 4.

4. Context window scaling yields diminishing returns. For any problem f, relevance density $\rho(f) = |K^{[f]}|/|\mathrm{ctx}(t)|$ decreases as context fills with content irrelevant to f (Definition 3). The prediction: increasing context window size while holding knowledge structure constant will show diminishing returns in task performance, and eventually negative returns for retrieval-sensitive tasks. The efficient scaling path is growing encoded knowledge accessible via stored adjacency, not growing the context window.

5. Retrieval-augmented systems outperform context stuffing. Direct consequence of Theorem 4. Retrieving the relevant subgraph $K^{[f]}$ and loading it into context maintains $\rho = 1$ regardless of total knowledge size. Stuffing all available context indiscriminately degrades ρ. The prediction: for any fixed context window, retrieval-augmented architectures will outperform context-stuffing architectures on knowledge-intensive tasks, with the gap widening as total knowledge grows.

6. Mature knowledge bases are edge-dense. Corollary 3 predicts that the ratio of edges to nodes increases as K matures under continuous evidence arrival. The prediction: empirical knowledge graphs (Wikidata, biomedical ontologies, institutional knowledge bases) will show increasing edge-to-node ratios over time, and the informational value of the graph (measured by downstream task performance) will correlate more strongly with edge density than with node count.

7. Error recovery cost scales with adjacency quality. The cascade-as-error-recovery remark predicts that the cost of recovering from corrupted evidence scales as $O(|D_p|)$ with stored typed adjacency and $\Omega(n)$ without it. The prediction: systems with explicit dependency tracking will recover from misinformation or evidence retraction faster and with less collateral damage than systems without it. Knowledge graphs that support typed cascade will outperform flat stores on correction tasks.

Biological evidence. Biological neural networks maintain sparse, structured connectivity rather than dense all-to-all connections. Working memory is bounded ($C_n \approx 7 \pm 2$) while long-term knowledge grows without apparent limit [18]. Retrieval is associative and context-driven, consistent with graph traversal from a query node rather than exhaustive scan. These observations are consistent with the theorems' predictions, though the biological case requires caution: the theorems assume propositional decomposition, and the degree to which biological representations admit such decomposition remains an open empirical question.

7 Conclusion

We have shown that graph structure over propositions is not a design choice but a necessary consequence of two independently sufficient constraints: information preservation under bounded capacity, and sub-linear knowledge maintenance under continuous evidence arrival. Continuous-weight mechanisms such as transformer self-attention implement graph structure dynamically rather

than constituting alternatives to it. The result applies to any system that accumulates propositions under bounded active context and must preserve information: such a system necessarily maintains a directed graph with typed edges, whether that graph is realized as explicit edges, learned sparse attention, synaptic connectivity, or any other medium.

Acknowledgments

The author used Claude (Anthropic) for drafting assistance, literature review, and LaTeX preparation. All theoretical content, proofs, and arguments are the author's own.

References

[1] R. Arp, B. Smith, and A. D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.

[2] F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, and P. F. Patel-Schneider, editors. *The Description Logic Handbook: Theory, Implementation, and Applications*. Cambridge University Press, 2003.

[3] T. Berners-Lee, J. Hendler, and O. Lassila. The semantic web. *Scientific American*, 284(5):34–43, 2001.

[4] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, volume 26, pages 2787–2795, 2013.

[5] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

[6] J. de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.

[7] J. Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.

[8] E. Dupoux, Y. LeCun, and J. Malik. Why AI systems don't learn and what to do about it: Lessons on autonomous learning from cognitive science. *arXiv preprint arXiv:2603.15381*, 2026.

[9] M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 5484–5495, 2021.

[10] N. Guarino. Formal ontology and information systems. In *Proceedings of FOIS'98*, pages 3–15. IOS Press, 1998.

[11] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. Ngonga Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[12] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

[13] K. Li, O. Press, and S. Arora. How does GPT-2 compute greater-than?: Interpreting mathematical abilities in a pre-trained language model. In *Advances in Neural Information Processing Systems*, 2024.

[14] C. Masolo, S. Borgo, A. Gangemi, N. Guarino, and A. Oltramari. WonderWeb deliverable D18: Ontology library (final). Technical report, ISTC-CNR, 2003.

[15] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[16] M. McCloskey and N. J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In G. H. Bower, editor, *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[17] K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372, 2022.

[18] G. A. Miller. The magical number seven, plus or minus two: Some limits on our capacity for processing information. *Psychological Review*, 63(2):81–97, 1956.

[19] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, 1988.

[20] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

[21] H. A. Simon. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1):99–118, 1955.

[22] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *Proceedings of the International Conference on Learning Representations*, 2019.

[23] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. In *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pages 368–377, 2000.

[24] W. A. Woods. What's in a link: Foundations for semantic networks. In D. G. Bobrow and A. Collins, editors, *Representation and Understanding: Studies in Cognitive Science*, pages 35–82. Academic Press, 1975.